

Para ver los GIFs:

<https://docs.google.com/presentation/d/1GoCTC0-qMVavRHtEPHxek2TEaA4B2WAILxS21wZsg3Y/edit?usp=sharing>

Métodos de Búsqueda Trabajo Práctico Nº1

Alicia Cobo Iglesias
Lucas Di Candia
Axel Farina
Emilio Mitchell
Tomas Pietravallo

5	7	3
8	2	
1	6	4

Ejercicio 1

8 Puzzle

3	2	1
6	5	4
	8	7

Ejercicio 1: 8 Puzzle, Estructura de Estado

Tablero 3×3, ocho fichas y un hueco · **costo uniforme** $c(s, a, s') = 1$ por movimiento

- Estado: tupla de 9 enteros —

5	7	3	8	2		1	6	4
---	---	---	---	---	--	---	---	---

 - Inmutable $\Rightarrow$ hashable $\Rightarrow$ visitados en $O(1)$
- Espacio: $9! = 362.880$ permutaciones, pero solo la mitad es alcanzable: $9! / 2 = 181.440$ estados.
- Operadores: intercambio del hueco con fichas adyacentes. Arriba, Abajo, Izquierda, Derecha (respetando bordes 3×3)

8-Puzzle: Paridad, solo una de las tres metas es alcanzable

Invariante: **paridad de inversiones**. No cambia nunca.

- Ancho impar $\Rightarrow$ alcanzable $\Leftrightarrow$ misma paridad

Inicial: 17 inversiones $\rightarrow$ impar

- Meta A $\rightarrow$ 0 (par) – inalcanzable
 - **Meta B $\rightarrow$ 7 (impar) – ALCANZABLE**
 - Meta C $\rightarrow$ 12 (par) – inalcanzable
- Dos de las tres metas del enunciado no tienen solución desde este tablero inicial
 - Se detecta contando inversiones, sin expandir un solo nodo

Tablero Inicial

5	7	3
8	2	
1	6	4

17 inv · impar

Meta A

1	2	3
4	5	6
7	8	

0 inv · par ✗

Rotación
90°

Meta C

3	6	
2	5	8
1	4	7

12 inv · par ✗
preserva la paridad

Espejo

Meta B

3	2	1
6	5	4
	8	7

7 inv · impar
✓
invierte la paridad

Heurísticas No-Triviales

h_1 : Distancia Manhattan Acumulada

Suma de distancias ortogonales desde la posición actual hasta la posición objetivo para cada una de las 8 fichas numeradas.

$$h_1(s) = \sum_{i=1}^8 (|x_i - x_i^{\text{goal}}| + |y_i - y_i^{\text{goal}}|)$$

Admisibilidad: Asume un problema relajado donde las fichas pueden deslizarse unas sobre otras sin bloquearse. El costo real nunca será menor a $h_1(s)$.

h_2 : Manhattan + Conflicto Lineal

Suma a la distancia Manhattan 2 movimientos adicionales por cada par de fichas en conflicto lineal.

$$h_2(s) = h_1(s) + 2 \times \text{ConflictosLineales}$$

Admisibilidad: Si dos fichas están en su fila/columna objetivo correcta pero invertidas, una debe abandonar la línea para permitir el paso de la otra, sumando obligatoriamente al menos 2 pasos reales.

Algoritmo recomendado: A*

Justificación Algorítmica

Función de Evaluación: Utiliza $f(n) = g(n) + h_2(n)$, combinando el costo real acumulado $g(n)$ con la estimación heurística $h_2(n)$.

Garantía de Optimalidad: Dado que el costo de cada paso es constante ($c = 1 > 0$) y h_2 es admisible y consistente, A* garantiza hallar el camino óptimo con el menor número de movimientos.

Viabilidad en RAM: Con un espacio de 181.440 estados posibles, toda la frontera de exploración y el conjunto de visitados entran holgadamente en la memoria principal si la memoria fuera acotada.

Métodos Descartados

BFS: óptimo con costo uniforme, pero $O(b^d)$ en memoria. Siendo “b” el Branching factor y “d” el Depth

DFS: no óptimo, cicla sin control de visitados.

Greedy: rápido, sin garantía de optimalidad.

8-Puzzle: método elegido y por qué

Heurística	$h(s_0)$	Costo hallado	Nodos expandidos	vs. mal ubicadas
Fichas mal ubicadas	8	23	12761	-
Manhattan	17	23	1049	12,2x
Manhattan + conflicto lineal	17	23	506	25,2x

A* contra la única meta alcanzable (Meta B). Costo óptimo $h^* = 23$ movimientos en los tres casos: las tres heurísticas son admisibles, cambia cuánto **cuesta llegar**.

A* con Manhattan + conflicto lineal

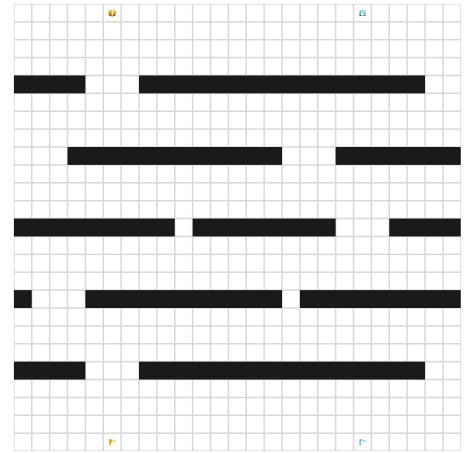
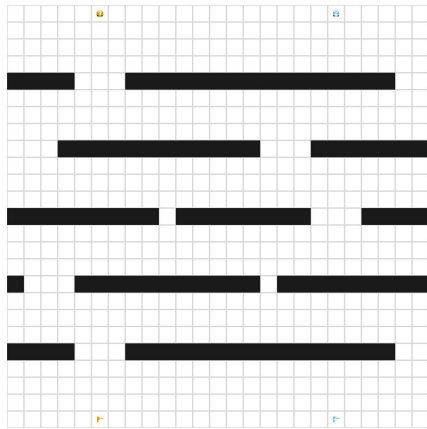
Admisible y **consistente** $\Rightarrow$ óptimo garantizado

25× menos expansiones que la heurística ingenua

Con memoria acotada: **IDA***, mismo óptimo en $O(d)$

Ejercicio 2

Grid World



Reglas de Movimiento y Validación de Estados Grid World

Rechazo de movimiento

- Fuera del mapa • pared • auto adelante • **bandera ajena**

Estacionamiento

- Al llegar a su meta el auto queda **PARKED** → obstáculo permanente

Casos borde

- **Parking order deadlock**: el orden puede bloquear irreversiblemente
- **Stranded cars**: flood-fill detecta estados perdidos → poda

Grid World como problema bien definido

Objetivo • cada auto sobre la bandera de su número, en la mínima cantidad de movimientos

- **Estado inicial:** posiciones iniciales de los N autos, ninguno estacionado
- $A = \{ \text{mover}(i, d) \mid i \in 1..N, d \in \{\text{arriba, abajo, izq, der}\} \}$
- $S = \{ s \mid s = (P, E) \}$ — P : posición de cada auto • E : conjunto de autos estacionados
- $\delta(s, a)$: mueve el auto i una celda en d , **si la celda destino es válida**; si llega a su bandera, lo agrega a E
- $\forall a \in A, g(a) = 1$ — costo uniforme
- $\text{goal}(s) == 1 \Leftrightarrow E = \{1, \dots, N\}$

El tablero (dimensiones, obstáculos, banderas) es fijo: no forma parte del estado.

Caracterización del ambiente

- **Totalmente observable** — el agente ve el tablero completo
- **Determinístico** — δ no tiene azar: una acción, un resultado
- **Secuencial** — cada movimiento condiciona los siguientes
- **Estático** — el tablero no cambia mientras el agente decide
- **Discreto** — celdas y movimientos finitos
- **Conocido** — las reglas se conocen de antemano
- **Individual** (*ver abajo*) · **no adversarial**

Sobre "multiagente"

- El enunciado llama al problema **Grid World multiagente** por los N autos. Pero en la taxonomía de ambientes es **individual**: hay **un solo agente** — el buscador — que decide qué auto mover.
- Los autos **no perciben ni deciden**: son entidades móviles bajo **control centralizado**.

Decisiones de diseño

Estado

- Board estático · GameState dinámico
- **Inmutable y hasheable** → conjunto de visitados
- Estacionados en **frozenset** → colapsa órdenes equivalentes

```
@dataclass(frozen=True, slots=True)
class GameState:
    cars: tuple[Position, ...]
    parked: frozenset[int]

    def position_of(self, car: int) -> Position:
        return self.cars[car - 1]

    def car_at(self, pos: Position) -> int | None:
        for index, car_pos in enumerate(self.cars):
            if car_pos == pos:
                return index + 1
        return None

    def is_parked(self, car: int) -> bool:
        return car in self.parked
```

```
@dataclass(frozen=True, slots=True)
class SearchNode:
    state: GameState
    parent: "SearchNode | None"
    action: LegalMove | None
    path_cost: int
    depth: int

    @classmethod
    def root(cls, state: GameState) -> "SearchNode":
        return cls(state=state, parent=None, action=None, path_cost=0, depth=0)

    def path(self) -> tuple[LegalMove, ...]:
        actions: list[LegalMove] = []
        node: SearchNode | None = self
        while node is not None and node.action is not None:
            actions.append(node.action)
            node = node.parent
        actions.reverse()
        return tuple(actions)
```

Decisiones de diseño

Algoritmos

- IDDFS: transposición **calificada por profundidad**, Se registra la profundidad mínima a la que se visitó cada estado dentro de la pasada actual
- Sucesores en orden determinista → reproducible
- **Graph search** en los cinco



```
if child_state in visited_depth and visited_depth[child_state] <= child_depth:  
    continue  
visited_depth[child_state] = child_depth
```

Esto poda caminos redundantes o más largos hacia un mismo estado, manteniendo la propiedad de encontrar la solución óptima en costo/pasos.

Medición

- 5 corridas por punto • **timeout 180 s**

Heurísticas

Las tres ignoran a los otros autos $\Rightarrow$ cotas inferiores

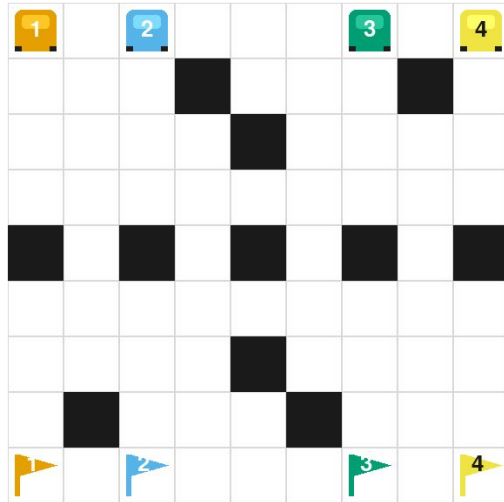
- **Manhattan (suma)** - un auto por turno $\Rightarrow$ los costos se acumulan
- **Max Manhattan** - descarta $N-1$ autos
- **Euclidiana** - $L2 \leq L1$, pero no hay diagonales

$h_{\text{euclidiana}} \leq h_{\text{manhattan}} \cdot h_{\text{max}} \leq h_{\text{manhattan}} \cdot \text{todas} \leq h^*$ ★

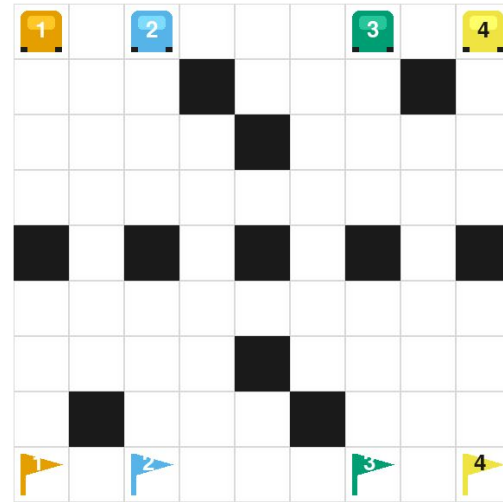
- En Classic, sobre un óptimo de 30: $h_{\text{max}} = 12 \leq h_{\text{euclid}} = 22 \leq h_{\text{manhattan}} = 30 = h^*$

Heurísticas

Manhattan (suma) – un auto por turno $\Rightarrow$ los costos se acumulan



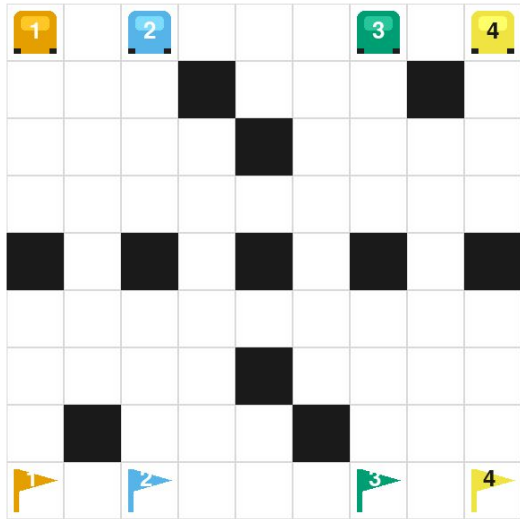
A^*
Manhattan



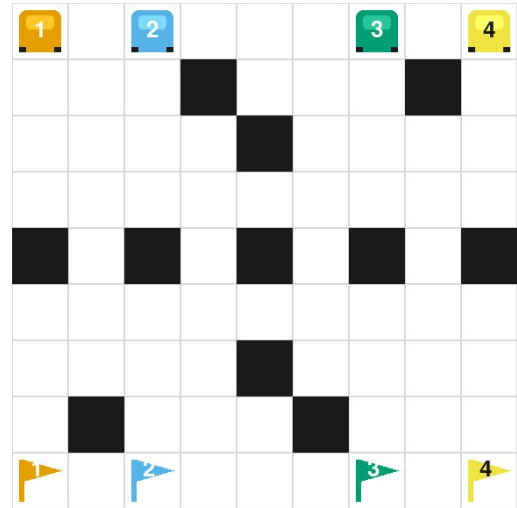
Greedy
Manhattan

Heurísticas

Max Manhattan — descarta N-1 autos



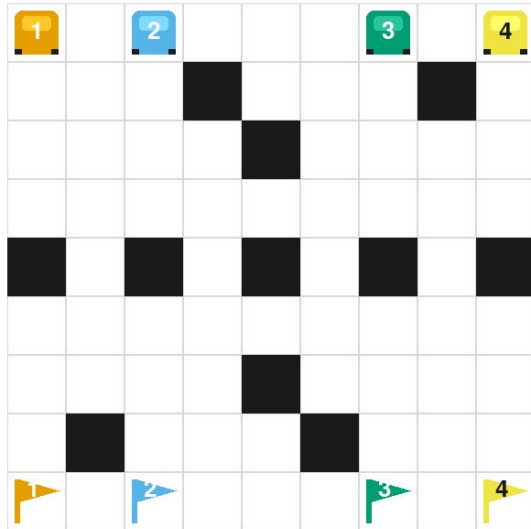
A*
Max Manhattan



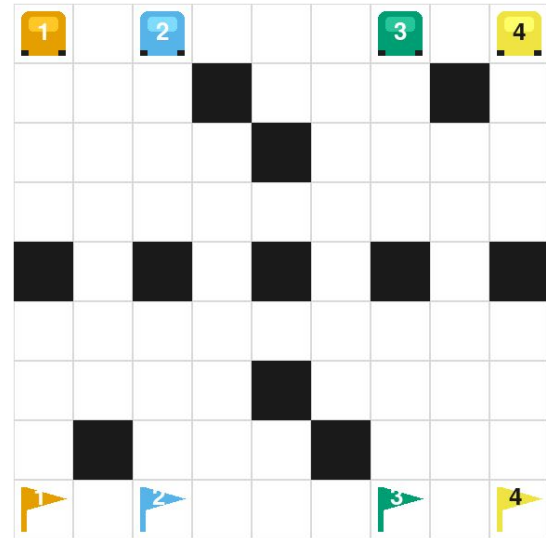
Greedy
Max Manhattan

Heurísticas

Euclidiana — $L2 \leq L1$, pero no hay diagonales



A* Euclidean



Greedy
Euclidean

Propiedades de las heurísticas

Admisible • $h(n) \leq h^*(n)$ — nunca sobreestima. Las tres lo son.

Consistente • $h(n_i) \leq c(n_i, n_{\square}) + h(n_{\square})$ y $h(n_g) = 0$

- Acá $c = 1$ y h cambia a lo sumo 1 por movimiento $\Rightarrow$ **se cumple**, así que A^* no reabre nodos ya expandidos

Dominancia • si h_2 domina a h_1 , A^*_2 expande $\leq$ que A^*_1

- $h_{\text{euclidiana}} \leq h_{\text{manhattan}}$ y $h_{\text{max}} \leq h_{\text{manhattan}}$ ✓
- h_{max} y $h_{\text{euclidiana}}$ **NO** son comparables — medido: en Marathon, $h_{\text{max}} > h_{\text{euclidiana}}$ en 40 de 92 pasos
- Combinación: $h' = \text{Max}(h_1, \dots, h_{\square})$ es admisible y domina a todas. Acá $h' = h_{\text{manhattan}}$, porque ya domina a las otras dos.

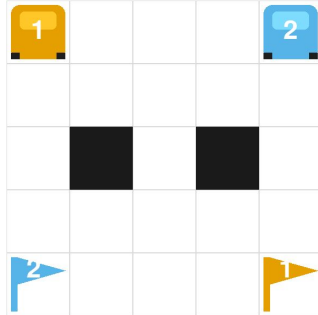
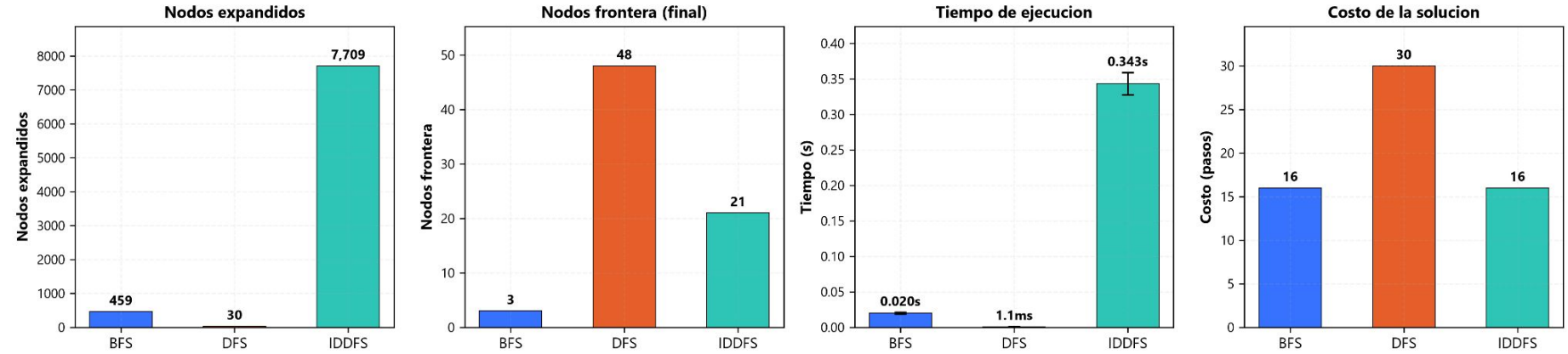
A^* es óptima acá • ramificación $\leq 4N$ finita • $g(a) = 1 > 0$ • h admisible

Resultados

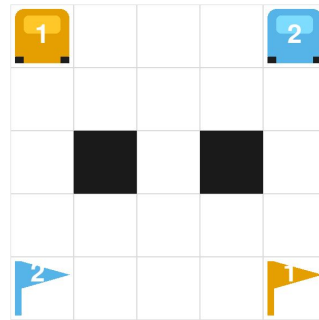
45 combinaciones · 5 niveles × (3 desinformados + 2 × 3 heurísticas)

- 41 completas · 4 timeouts de 180 s
 - BFS → Gridlock
 - IDDFS → Classic, Gridlock, Marathon
- Ningún método informado agotó el límite.
- Mini 3×3·2 → Warm Up 5×5·2 → Classic 7×7·3 → Gridlock 9×9·4 → Marathon 25×25·2
- Camino solución en `SearchResult.path`, reproducido en los GIFs

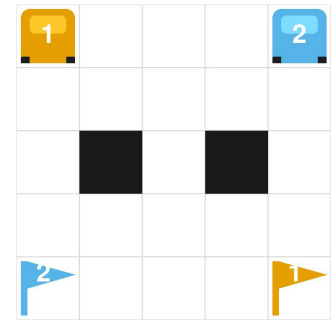
Modelos Desinformados – Level: Warmup



BFS

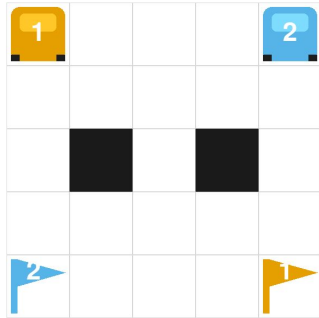
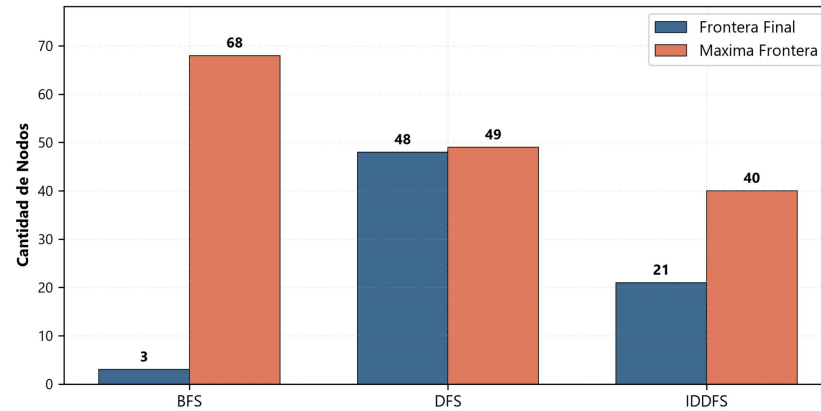


DFS

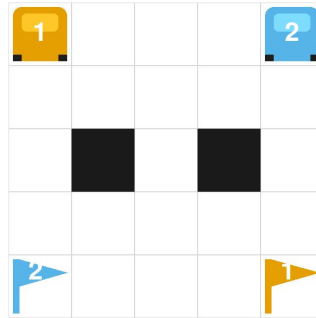


IDDFS

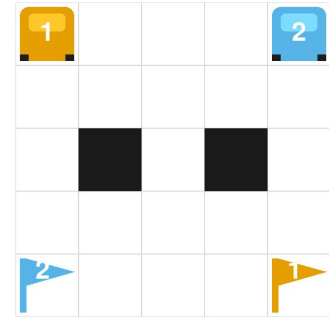
Nodos Frontera: Cantidad Maxima vs Final – Level: Warm Up



BFS

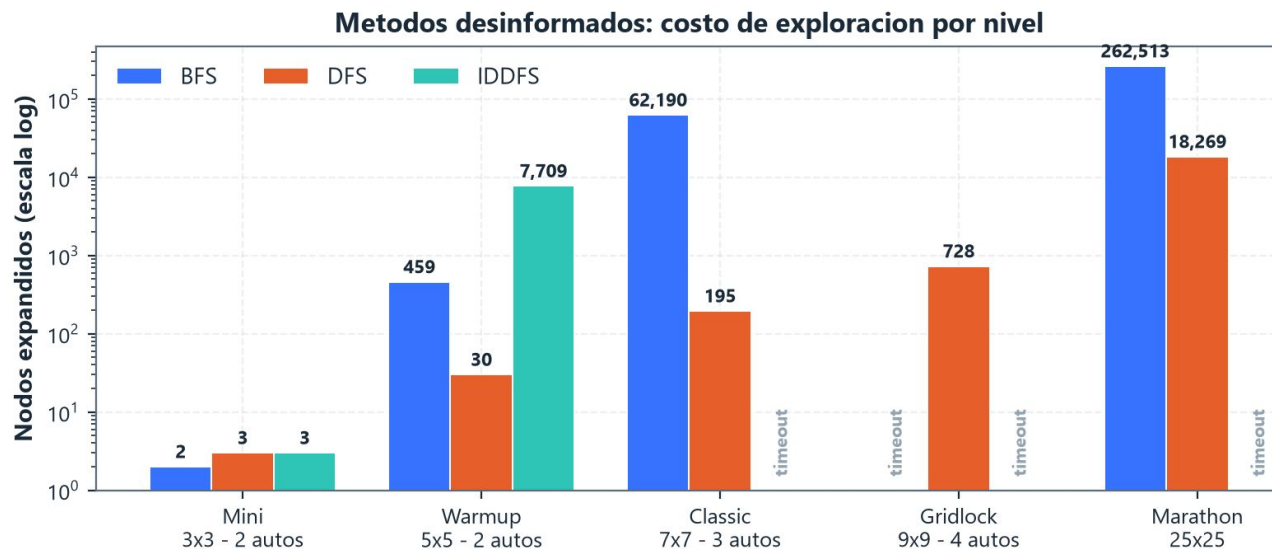


DFS



IDDFS

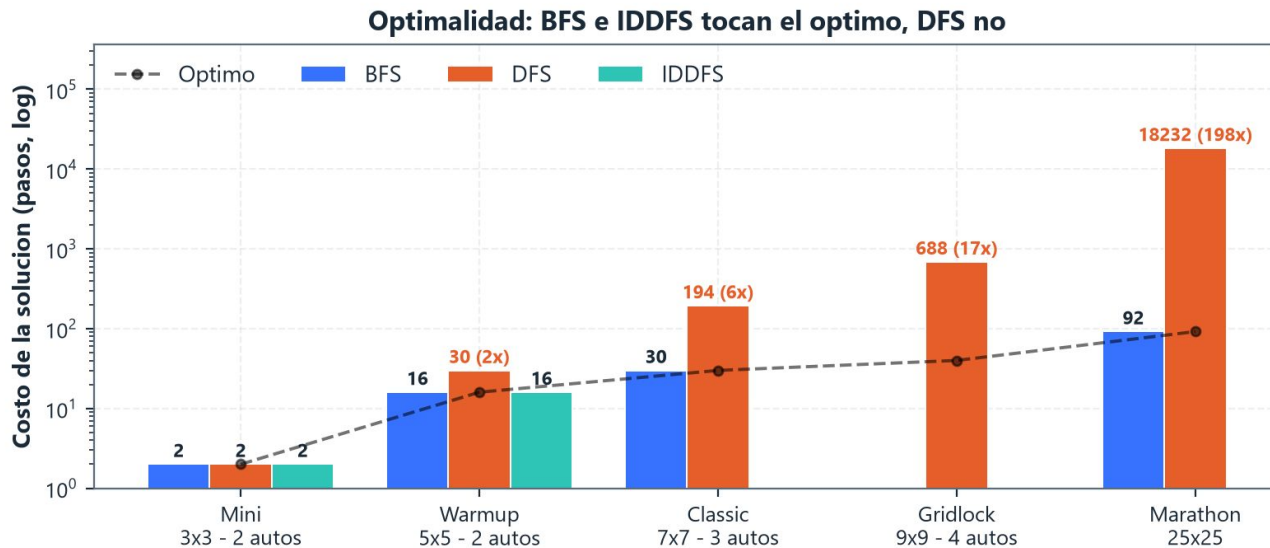
Desinformados: cuánto explora cada uno



IDDFS: 16,8× los nodos de BFS, a cambio de 40 % menos memoria. Escala logarítmica.

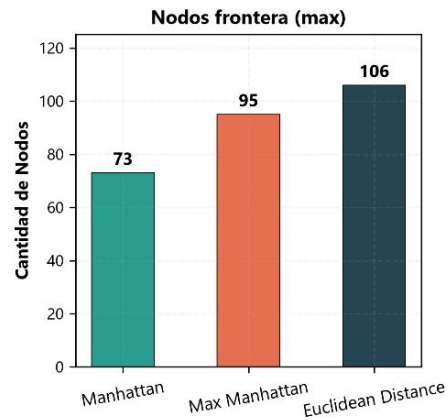
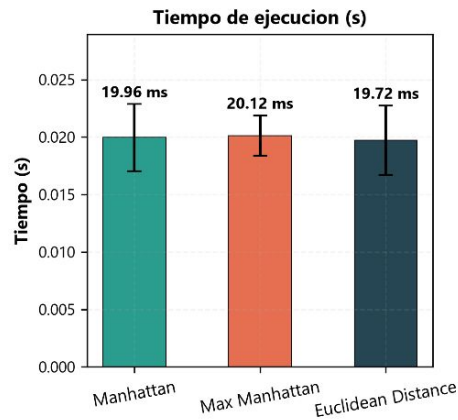
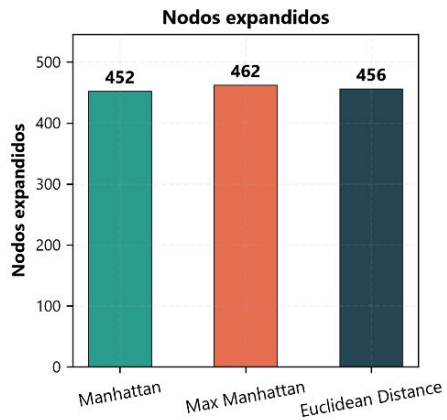
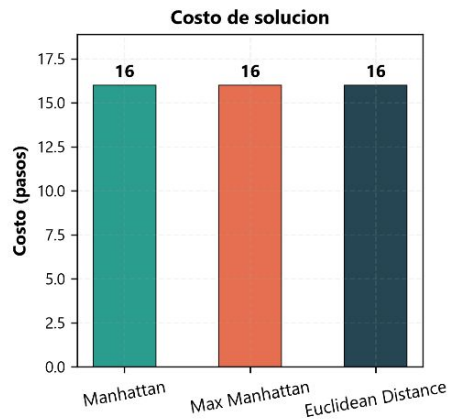
timeout (180 s)

Desinformados: el costo de la solución hallada



DFS: $6\times \cdot 17\times \cdot 198\times$ peor que el óptimo. Su bajo conteo es rendición, no eficiencia.

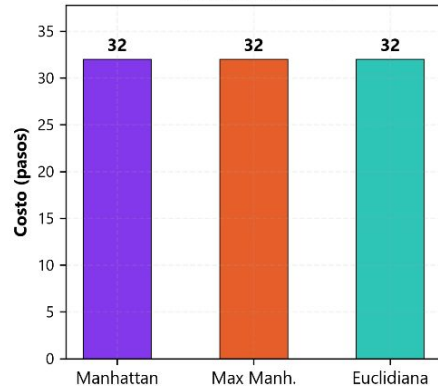
Comparativas de Heurísticas A* - Level: Warm Up



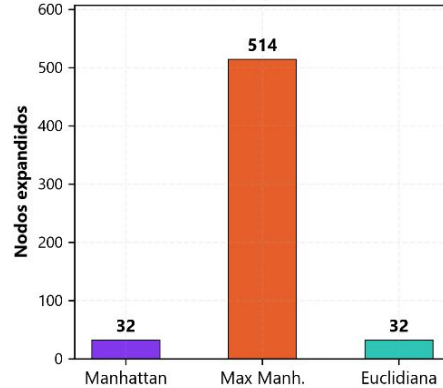
Warm Up ($5 \times 5 \cdot 2$ autos). Mismo óptimo de 16 pasos y prácticamente el mismo esfuerzo: 452 vs 462 nodos (2%). El nivel es demasiado chico para separar heurísticas, el espacio tiene 484 estados (aprox.) y A* expande casi todo con cualquiera de las tres. La brecha real aparece en Classic ($3,2 \times$), en la lámina de Greedy vs A*.

Comparativas Heurísticas de Greedy

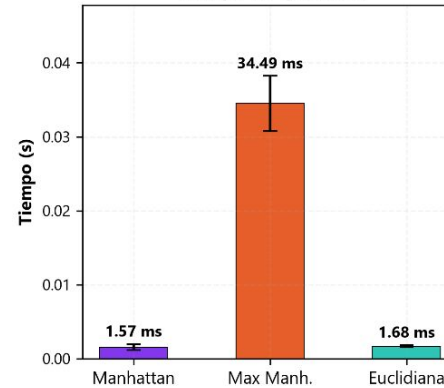
Costo de solucion



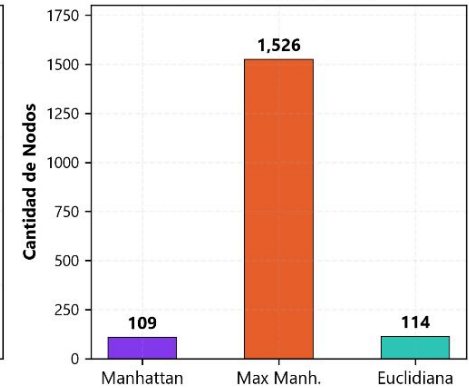
Nodos expandidos



Tiempo de ejecucion

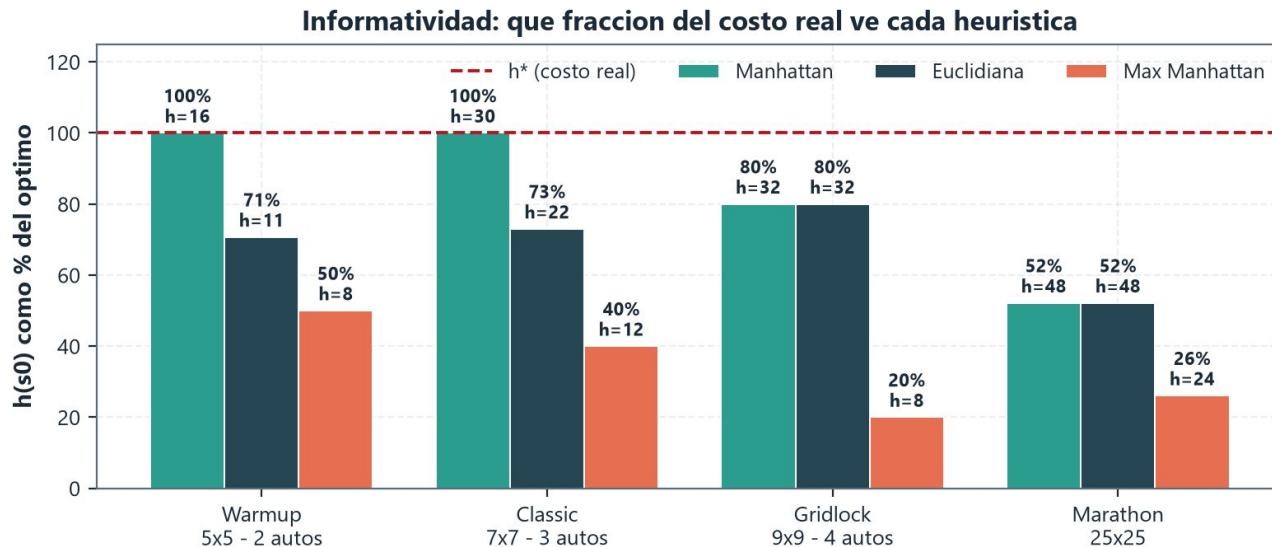


Nodos frontera (max)



Sin `g` que lo corrija, Greedy **amplifica** la heurística: **32 vs 514** nodos.

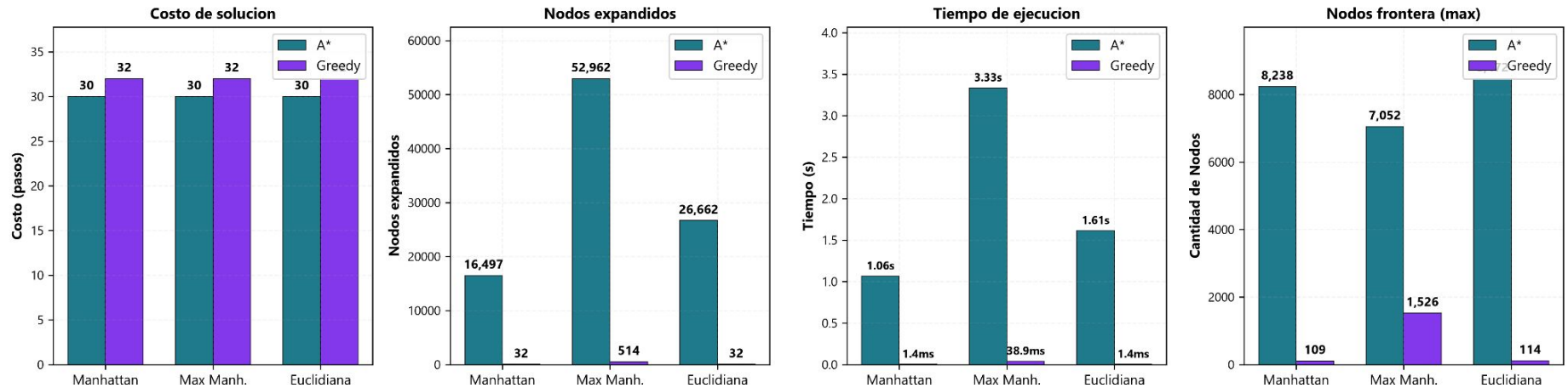
Informatividad: cuánto sabe cada heurística



Manhattan es **exacta en la raíz** en Classic — y aun así A* expande 16.497 nodos. **h se degrada camino adentro.**

Greedy vs A*

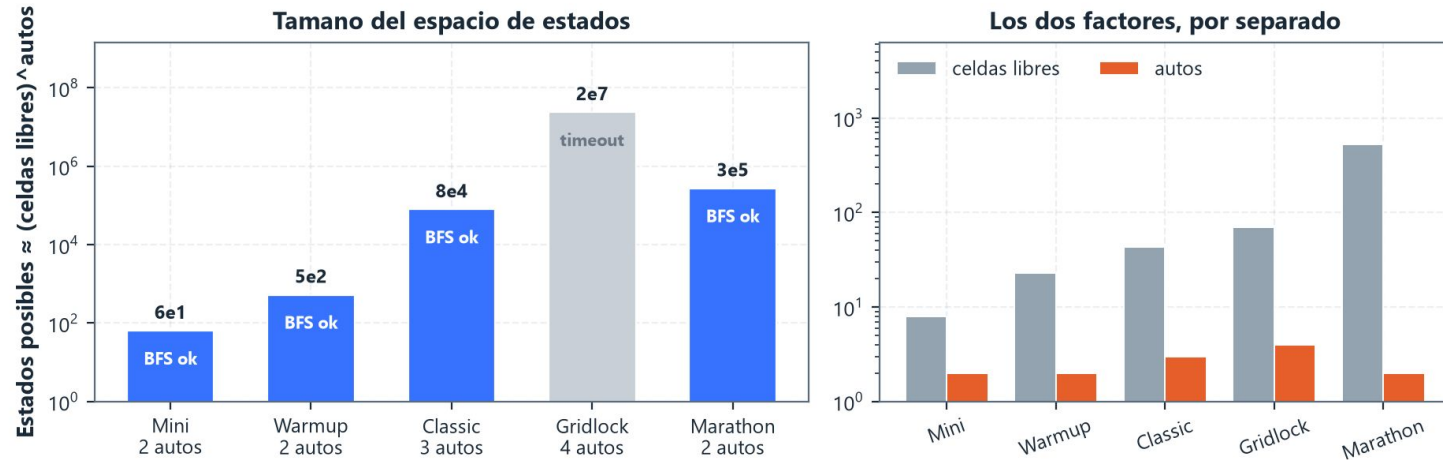
Comparativa de Heurísticas en Algoritmos Informados (A* vs Greedy) - Classic



516× menos nodos, +2 pasos. No es óptimo porque ignora `g`, no por la heurística.

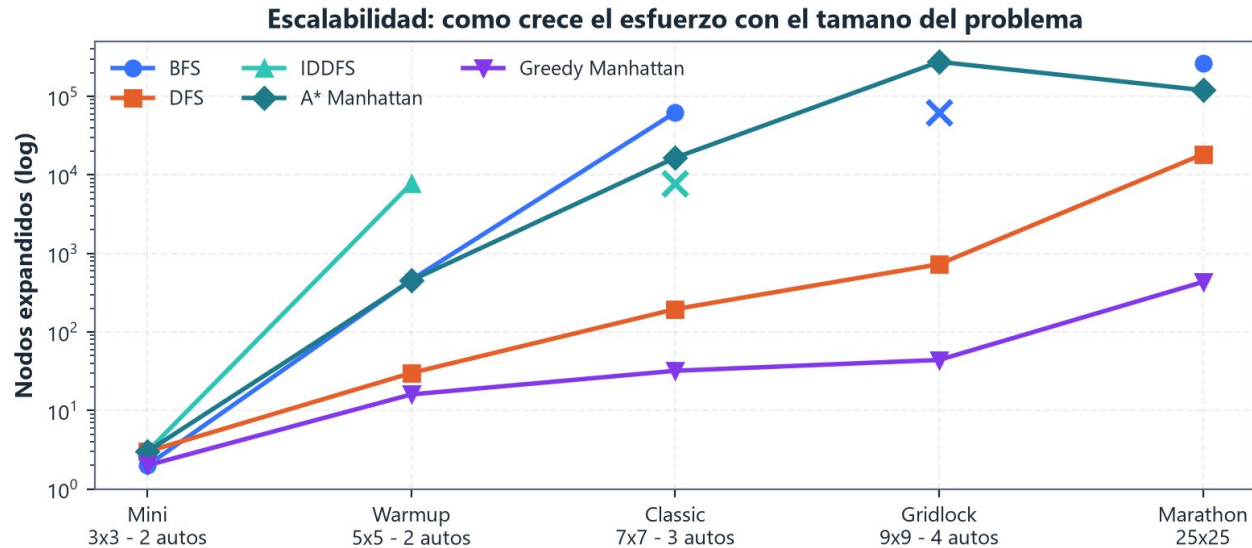
Qué hace difícil a un tablero

Lo que rompe la búsqueda son los agentes, no el tablero



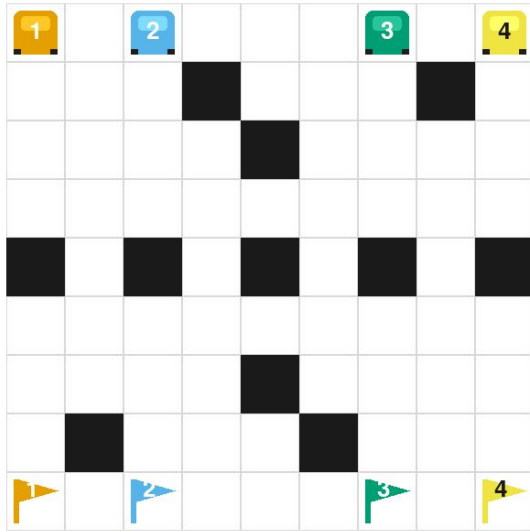
Un 25×25 con 2 autos se resuelve; un 9×9 con 4 no. Espacio $\approx$ (celdas libres)^autos.

Escalabilidad: cómo crece el esfuerzo

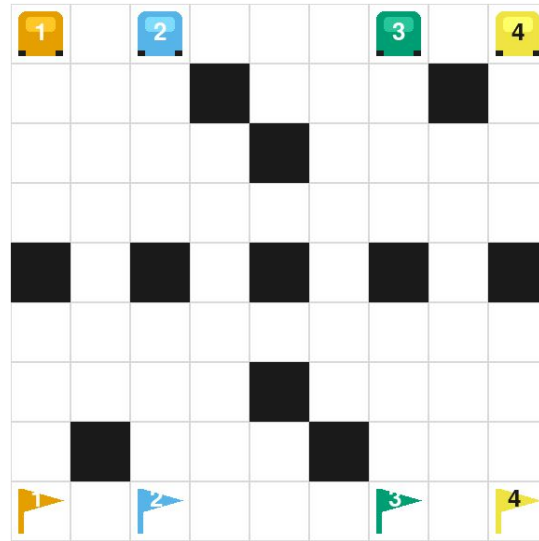


X = cortado a 180 s. La posición del marcador no representa nodos.

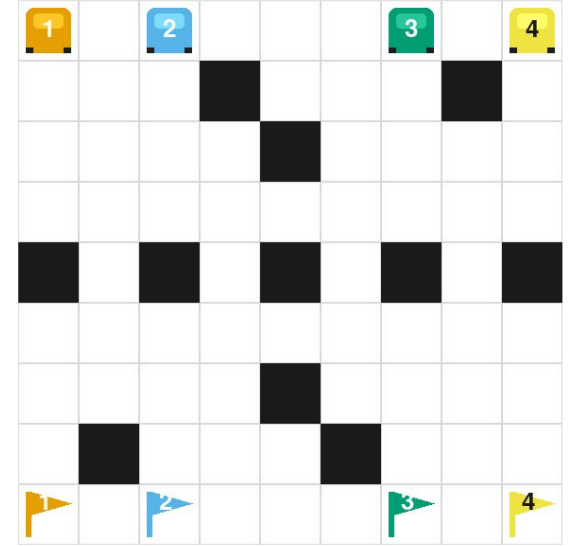
A* Heuristics – Level: Classic



A*
Euclidian

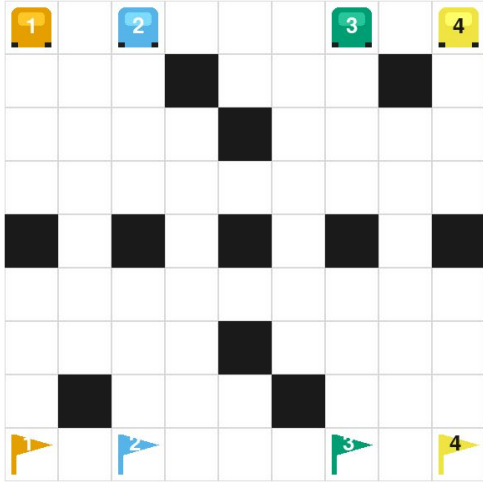


A*
Manhattan

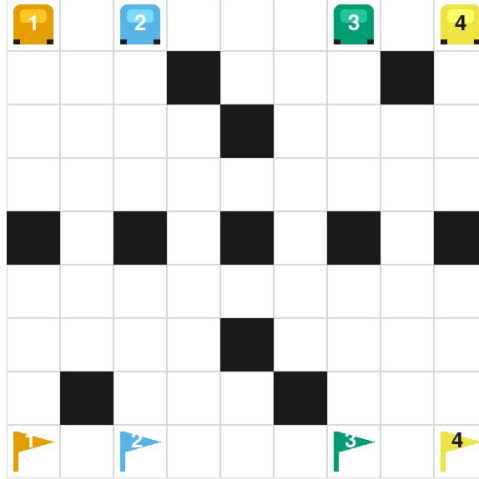


A*
Max Manhattan

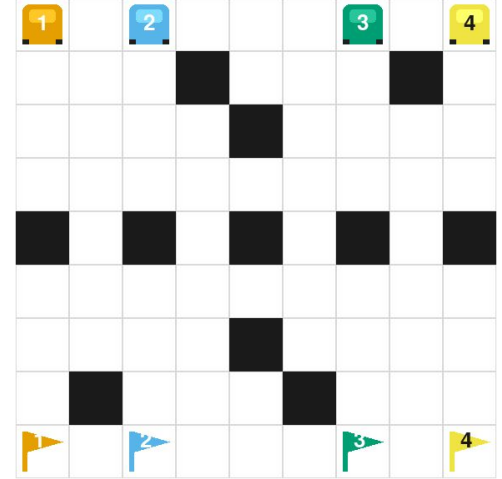
Greedy Heuristics – Level: Classic



Greedy
Euclidean

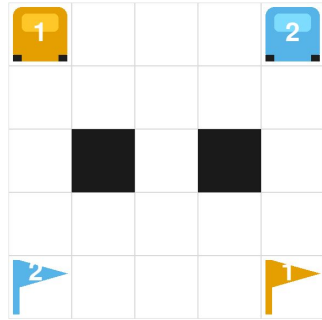
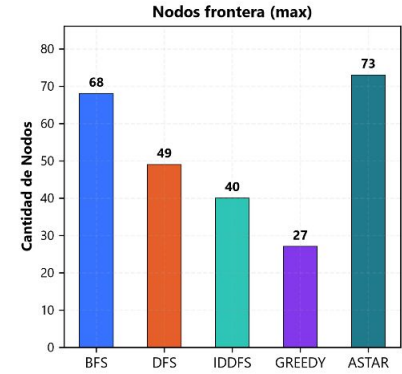
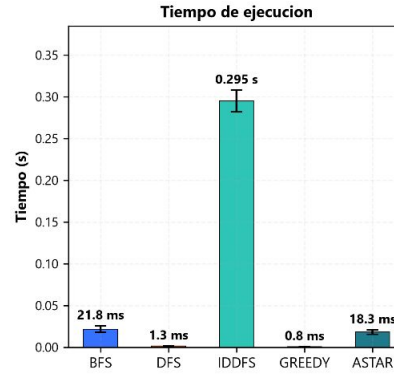
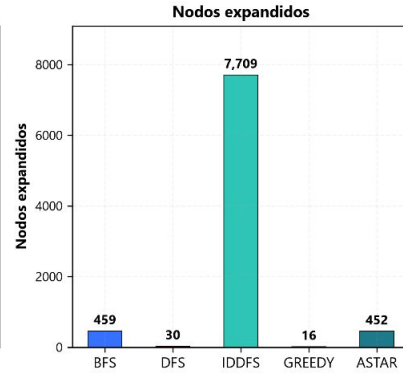
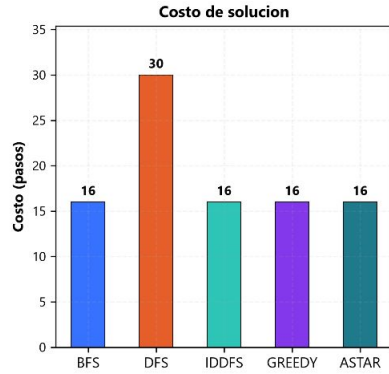


Greedy
Manhattan

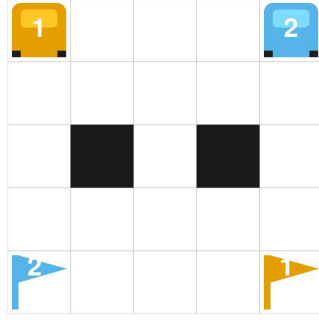


Greedy
Max Manhattan

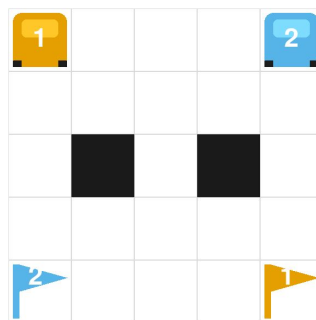
Level: Warm Up



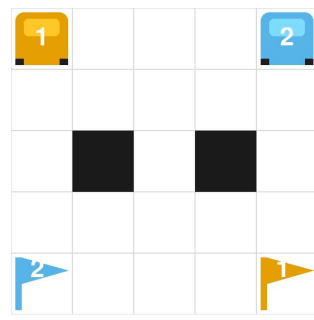
BFS



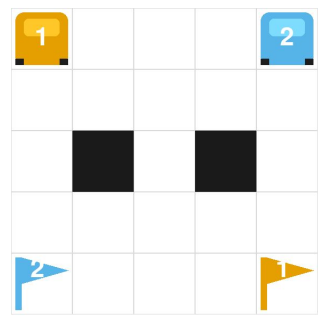
DFS



IDDFS

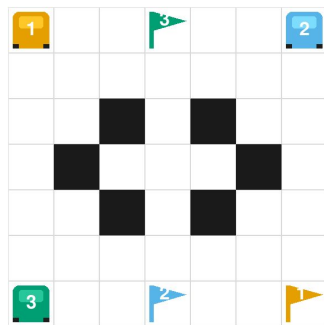
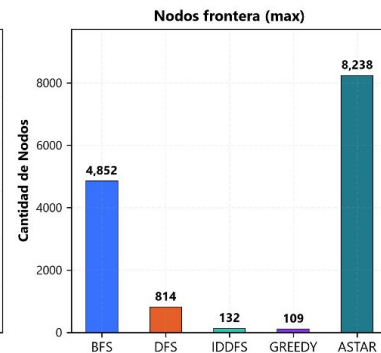
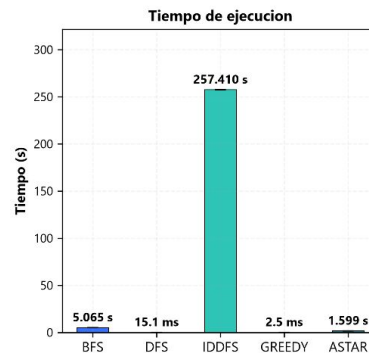
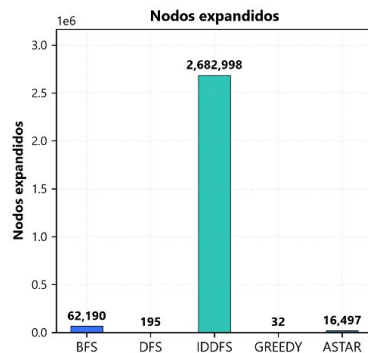
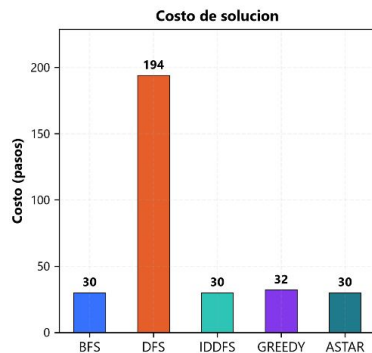


GREEDY

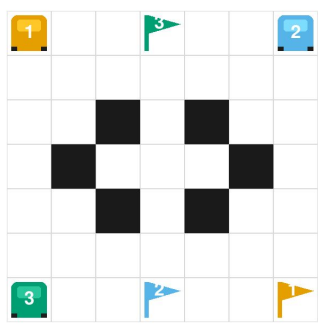


A*

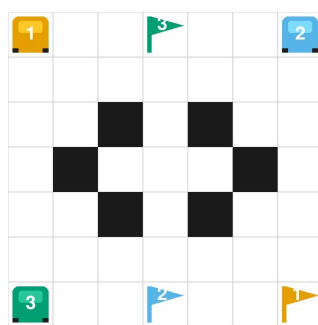
Level: Classic



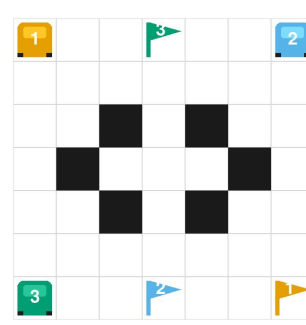
BFS



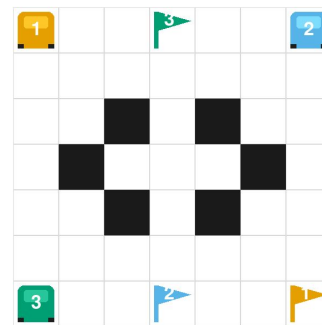
DFS



IDDFS



GREEDY



A*

IDDFS en Classic se corrió sin límite de tiempo: 257 s, 2.662.998 nodos. Bajo el timeout de 180 s del resto de las mediciones no termina, por eso figura como timeout en las láminas comparativas.

Conclusiones — desinformados e informados

Desinformados

- BFS e IDDFS: **óptimos**, y caros por eso mismo
- BFS muere en el 9×9 . IDDFS también. Cambia $17 \times$ nodos por -40% de memoria (Warm Up), pero no alcanza para escalar.
- DFS: hasta **$198 \times$** peor que el óptimo

Informados

- Misma admisibilidad, mismo óptimo, **precio distinto**
- Classic: **16.497** (Manhattan) vs **52.962** (Max) vs 62.190 (BFS)
- Heurística floja $\Rightarrow A^* \approx \text{BFS}$, con el costo de evaluarla

Conclusiones — qué elegir, y de qué depende

Sin condiciones

- Informados $\gg$ desinformados, y la brecha **crece con el problema**

Todo lo demás depende

- Del **modelado**: suma vs máximo
- De los **agentes**, no del tamaño del tablero
- Del **objetivo**: costo / tiempo / memoria / nodos

Regla práctica

- Óptimo demostrable $\rightarrow A^* +$ la **h** admisible más informada
- Solución ya, tablero grande $\rightarrow$ **Greedy** (516× menos, +2 pasos)

**Muchas
gracias**

¿Preguntas?